

信息与知识获取作业三实验报告

李浩薪

日期：2023 年 5 月 31 日

1 引言

1.1 背景和意义

在当今数字化时代，海量的文本和数据被创建和存储，如何从中获取有价值的信息变得至关重要。信息抽取是一项涉及自然语言处理和机器学习的关键任务，旨在从非结构化文本中自动提取结构化的、有意义的信息。信息抽取技术可用于从新闻文章、科技论文、社交媒体等各种文本源中获取实体、关系、事件等信息，为信息检索、数据分析和决策支持等领域提供重要的支持。设计和实现一个信息抽取实验系统具有以下重要意义：

- 实践应用自然语言处理技术：通过开发信息抽取实验系统，可以应用自然语言处理技术，包括分句、分词、命名实体识别、句法分析等基本模块，加深对自然语言处理算法和技术理解和掌握。
- 提取特定领域的有价值信息：通过选择特定领域的语料，我们可以针对自己感兴趣的特定信息点进行抽取。这使得系统能够在特定领域的文本中识别和提取关键信息，为相关领域的研究、决策或其他应用提供支持。
- 算法评估与改进：实验过程中，可以对信息抽取算法进行评估，包括准确率、召回率等指标的计算。这有助于评估算法的性能，并为改进和优化提供依据。

通过实现一个信息抽取实验系统，我们可以深入了解信息抽取领域的相关技术和方法，并为实际应用和进一步研究提供有益的经验 and 见解。

1.2 作业要求和扩展要求

作业要求：

1. 设计并实现一个信息抽取实验系统，可以处理中文和英文文本。
2. 可以基于作业 2 信息检索系统进行扩展，也可以独立实现。
3. 选定一个特定领域的语料，并进行本地存储。语料规模不低于 100 篇文档。
4. 选择自己感兴趣的特定信息点进行抽取，信息点的个数不低于 5 个。
5. 可以调用开源的中英文自然语言处理基本模块，如分句、分词、命名实体识别、句法分析等。
6. 实现信息抽取算法，至少包括正则表达式匹配算法的特定信息点抽取。
7. 最好能对抽取结果的准确率进行人工评价。
8. 界面不作强制要求，可以是命令行，也可以是可操作的界面。

9. 提交作业报告和源代码。

扩展要求：

1. 鼓励有兴趣和能力的同学积极尝试多媒体信息抽取。
2. 鼓励优化各模块算法，并关注相关竞赛。
3. 自主开展相关文献调研与分析，完成算法评估、优化和论证创新点的过程。

通过满足作业要求，我们将能够设计和实现一个具有信息抽取功能的实验系统，并针对特定领域的文本进行信息点的抽取。

2 相关工作和方法分析

2.1 相关工作

信息抽取是一个广泛研究的领域，许多研究者和机构已经提出了各种各样的方法和算法。以下是一些与信息抽取相关的工作：

1. 基于规则的信息抽取：这种方法依赖于手工编写的规则和模式，用于从文本中识别和提取特定信息。规则可以基于词汇、句法结构、正则表达式等。然而，这种方法的局限性在于需要大量的人工工作，并且难以应对复杂的抽取任务。
2. 基于机器学习的信息抽取：这种方法利用机器学习算法从标记好的训练数据中学习抽取模型。常见的机器学习算法包括条件随机场（CRF）、支持向量机（SVM）和深度学习模型等。机器学习方法可以自动从数据中学习特征和规律，并具有适应不同领域和任务的能力。
3. 远程监督信息抽取：这种方法结合了规则和机器学习的优势，利用知识图谱等外部知识源，将标记的实体和关系对应到未标记的文本中。远程监督可以自动为训练数据生成标签，从而加速信息抽取的模型训练过程。
4. 深度学习在信息抽取中的应用：深度学习模型如卷积神经网络（CNN）和循环神经网络（RNN）等已被广泛用于信息抽取任务。这些模型可以自动从原始文本中学习到有用的特征表示，提高信息抽取的性能。

2.2 现有处理方法

频率提取方法 频率提取方法是一种基于统计的信息抽取方法，它依赖于文本中词语或模式的频率信息来进行抽取。这种方法通常适用于特定类型的信息点抽取，其中信息点在文本中以相对固定的形式出现。其优点是易于实现和理解，不需要过多的复杂算法或训练过程并且可以应用于各种文本类型和领域，只需要关注关键词或模式的出现频率即可。但其同样有着上下文限制以及依赖于对领域知识的了解和关键词的选择的缺点。

自然语言处理（NLP） 自然语言处理是一种基于机器学习和深度学习的方法，利用自然语言处理技术来抽取文本中的信息。这种方法结合了语言学、统计学和计算机科学的知识，能够自动学习文本的特征和规律。其可以通过分句、分词、词性标注、句法分析等技术对文本进行深入分析，理解词语之间的关系和上下文信息，从而提高抽取的准确性，也可以在不同领域和文本类型中应用，通过学习通用的语言模型和特征表示来抽取信息，具有较强的泛化能力。但其

通常需要大量的标注好的训练数据并且涉及到复杂的模型和算法，需要较高的计算资源和时间成本来训练和推断模型。

3 系统设计与实现

3.1 整体设计思路和架构

前端设计与实现 使用 React 框架进行前端开发，搭建用户界面。设计搜索框、展示搜索结果的列表等组件，用户输入查询关键词后触发搜索事件并且点击结果中的某篇文章后将会跳转到文章对应的页面。

后端设计与实现 使用 Flask 框架搭建后端服务器，处理前端请求并返回相应的数据。

3.2 选取的数据集和本地存储方式

通过 CNN news 数据集，将 40000+ 条新闻存储到本地的 csv 文件中，经过 python Pandas 对 csv 文件进行处理，得到包含 summary 以及 key words 的 json 文件。

3.3 数据预处理

使用 Spacy 进行分词和实体提取。

3.4 信息抽取

3.4.1 Sequence to Sequence

在实验开始过程中，我最初是通过构建 Seq2Seq 模型来实现得到文本的 summary，我先使用了 Keras 库来构建和训练这个模型，定义了三个 LSTM 层，每个层都返回序列输出和隐藏状态，将第一个 LSTM 层的输出作为第二个 LSTM 层的输入，最后一个 LSTM 层的输出作为编码器的输出，并获得最终的隐藏状态 state_h 和 state_c。

```
# 编码器
encoder_inputs = Input(shape=(max_text_len,))
enc_emb = Embedding(x_voc, embedding_dim, trainable=True)(encoder_inputs)
encoder_lstm1 = LSTM(latent_dim, return_sequences=True, return_state=True, dropout
    =0.4, recurrent_dropout=0.4)
encoder_output1, state_h1, state_c1 = encoder_lstm1(enc_emb)
encoder_lstm2 = LSTM(latent_dim, return_sequences=True, return_state=True, dropout
    =0.4, recurrent_dropout=0.4)
encoder_output2, state_h2, state_c2 = encoder_lstm2(encoder_output1)
encoder_lstm3 = LSTM(latent_dim, return_state=True, return_sequences=True, dropout
    =0.4, recurrent_dropout=0.4)
encoder_outputs, state_h, state_c = encoder_lstm3(encoder_output2)
```

同时定义如下的解码器，将目标摘要嵌入到低维空间中，得到嵌入向量 `dec_emb`，定义一个 LSTM 层返回序列输出和隐藏状态并将 LSTM 层的输出应用于全连接层，使用 `softmax` 激活函数生成最终的输出。

```
# 解码器
decoder_inputs = Input(shape=(None,))
dec_emb_layer = Embedding(y_voc, embedding_dim, trainable=True)
dec_emb = dec_emb_layer(decoder_inputs)
decoder_lstm = LSTM(latent_dim, return_sequences=True, return_state=True, dropout
                    =0.4, recurrent_dropout=0.2)
decoder_outputs, decoder_fwd_state, decoder_back_state = decoder_lstm(dec_emb,
                              initial_state=[state_h, state_c])
decoder_dense = TimeDistributed(Dense(y_voc, activation='softmax'))
decoder_outputs = decoder_dense(decoder_outputs)
```

随后将编码器和解码器组合成一个模型，并指定优化器（`rmsprop`）和损失函数（稀疏分类交叉熵）来编译模型并随后进行训练（如果验证损失不再改善，则提前停止训练）。

```
model = Model([encoder_inputs, decoder_inputs], decoder_outputs)
model.summary()
model.compile(optimizer='rmsprop', loss='sparse_categorical_crossentropy')

es = EarlyStopping(monitor='val_loss', mode='min', verbose=1, patience=2)

history = model.fit([x_tr, y_tr[:, :-1]], y_tr.reshape(y_tr.shape[0],
              y_tr.shape[1], 1)[:, 1:], epochs=100, callbacks=[es], batch_size=512,
              validation_data=([x_val, y_val[:, :-1]], y_val.reshape(y_val.shape[0],
              y_val.shape[1], 1)[:, 1:]))
```

由于设备的限制，我们将训练挂载到 `kaggle` 上进行，通过 P100 接近 10 小时的训练，最终得到我们的 `model`，但是遗憾的是在进行测试阶段时其并不可很好的进行 `summary`，有很大一部分时间甚至无法给出一个完整的句子，只能给出几个零散的单词。

	R-1	R-2	R-L	R-AVG
Bottom-Up Sum	41.22	18.68	38.34	32.75
Lede-3	40.38	17.66	36.62	31.55
Seq2Seq + Attn	31.33	11.81	28.83	23.99
GPT-2 TL; DR:	29.34	8.27	26.58	21.40
Random-3	28.78	8.63	25.52	20.98
GPT-2 no hint	21.58	4.03	19.47	15.03

Table 4. Summarization performance as measured by ROUGE F1 metrics on the CNN and Daily Mail dataset. Bottom-Up Sum is the SOTA model from (Gehrmann et al., 2018)

在“Language models are unsupervised multitask learners” [1] 一文中，GPT2 同样针对 CNN

daily news 进行了 summary，但是结果是其只是勉强胜过从文本中选择 3 个随机句子，相比直接提取文章的前三句话，GPT2 的准确性还要差上很多。

因此，由于设备以及训练中对数据集处理要求的限制，我们最终不得不放弃使用 Seq2Seq 处理，转而采取基于出现频率以及位置的 Luhn 算法来获取 summary。

3.4.2 Luhn

对于每篇文章，使用 Spacy 进行实体提取并统计每一个实体出现的次数，将获取到的实体保存在最终的 json 文件当中。并通过 Luhn 算法根据关键词的频率来确定关键句，将这些句子组合成文章的摘要保存在最终的 json 文件中。其中 Spacy 所提取的实体类型包括如下字段：

PERSON (人名)：	具体的人物名称，如 "John Smith"
NORP (民族、宗教或政治团体)：	民族、宗教、政治团体等名称，如 "Chinese"
FAC (建筑、机构、机场等)：	建筑、机构、机场等的名称，如 "Empire State Building"
ORG (组织机构)：	组织机构的名称，如 "Google"、"United Nations"
GPE (地理政治实体)：	地理政治实体的名称，如 "New York"、"France"
LOC (位置)：	具体的位置名称，如 "park"、"beach"
PRODUCT (产品)：	产品的名称，如 "iPhone"、"Coca-Cola"
EVENT (事件)：	具体的事件名称，如 "Olympic Games"、"birthday party"
WORK_OF_ART (艺术作品)：	艺术作品的名称，如 "Monalisa"、"Hamlet"
LANGUAGE (语言)：	具体的语言名称，如 "English"、"Spanish"
DATE (日期)：	具体的日期，如 "2022-01-01"、"October 10th"
TIME (时间)：	具体的时间，如 "9:00 AM"、"15:30"
MONEY (货币)：	具体的货币金额，如 "\$100"、"¥500"

为了实现 Luhn 的算法，我们设计了一个名为 TextSummarizer 的类。在 TextSummarizer 类的构造函数中，可以设置几个参数来控制算法的行为，其中 top_n_words 参数表示要考虑的最常见词的数量，distance 参数表示词之间的最大距离，number_phrases 参数表示要返回的摘要句子的数量。实施预处理方法以预处理输入文本。它将文本转换为小写，并使用 NLTK 库 Split Text to Word 中的 word_tokenize 函数。接下来过滤停用词（比如英文常用词）和标点符号，过滤掉数字。最后，将处理后的词组装成格式化的文本字符串。最后，执行计算 _sentence 的 _score 方法用于计算句子的分数。它以一个短语列表和一个重要单词列表作为输入。对于每一个句子，它存储重要词在句子中的索引位置在 _word 列表索引中，并按索引位置排序。然后根据给定的距离参数将这些索引位置拆分到不同的桶中。对于每个组，它会计算组分数并保存最高分。最后，它返回一个包含句子分数和句子索引的列表。

核心算法代码如下，具体解释已由注释形式给出：

```
def calculate_sentence_score(self, sentences, important_words):  
    # 存储每个句子的得分以及句子的索引  
    notes = []  
    # 句子的索引  
    indice_sentence = 0  
    # 对每个句子进行处理  
    # 首先将每个句子转换为小写，使用nltk库的word_tokenize方法将句子分词为单词列表
```

```

for sentence in [nltk.word_tokenize(sentence.lower()) for sentence in sentences
]:
    indice_word = []
    # 获取句子中重要单词的索引位置
    for word in important_words:
        try:
            indice_word.append(sentence.index(word))
        except ValueError:
            pass
    # 确保索引位置按升序排列
    indice_word.sort()
    # 句子中没有重要单词，跳过
    if len(indice_word) == 0:
        continue
    # 存储根据距离将索引位置分组后的组
    list_groups = []
    group = [indice_word[0]]
    i = 1
    # 根据距离将索引位置分组
    while i < len(indice_word):
        if indice_word[i] - indice_word[i - 1] < self.distance:
            # 将当前索引位置添加到group列表中
            group.append(indice_word[i])
        else:
            # 将当前的group列表添加到list_groups列表
            list_groups.append(group[:])
            # 创建一个新的group列表
            group = [indice_word[i]]
        i += 1
    # 将最后的group列表添加到list_groups列表
    list_groups.append(group)
    # 存储最高得分的分组得分
    high_grade_group = 0
    # 计算每个分组的得分
    for g in list_groups:
        important_words_in_the_group = len(g)
        total_word_in_group = g[-1] - g[0] + 1

        note = 1.0 * important_words_in_the_group ** 2 / total_word_in_group

        if note > high_grade_group:
            high_grade_group = note

    notes.append((high_grade_group, indice_sentence))
    indice_sentence += 1

return notes

```

在调用 TextSummarizer 只需调用 summarize 接收原始文本作为输入即可, summarize(text) 将按以下步骤生成摘要:

1. 将文本分成原始句子列表。
2. 对每个原始句子进行预处理, 得到格式化句子列表。
3. 将格式化句子列表中的所有单词合并到一个单词列表中。
4. 使用 NLTK 库的 FreqDist 函数计算单词的频率分布。
5. 选择频率最高的 top_n_words 个单词作为重要单词列表。
6. 调用 calculate_sentence_score 方法计算句子得分。
7. 选择得分最高的 number_sentences 个句子作为摘要的一部分。
8. 返回摘要句子。

3.4.3 match rate

要得到最终的 match rate, 我们不仅考虑了搜索词 (query) 与 summary 的匹配程度, 同时还考虑了 query 中经过分词后与 Spacy 提取 text 中的 Top tokens 的匹配程度, 并通过一定的计算公式来得到最终的 match rate。

其中, 与 summary 的匹配程度的衡量我们首先使用向量化器 vectorizer 的 fit_transform 方法将 documents 列表中的文档 (query and summary) 转换为 TF-IDF 特征向量表示, 得到一个 TF-IDF 矩阵 tfidf_matrix。然后用余弦相似度 (cosine_similarity) 计算 query 与我们得到的 summary 之间的相似度矩阵。

```
def calculate_match_rate(self, keywords, summary):
    summary = summary.lower() # 将摘要转换为小写
    keywords = [keyword.lower() for keyword in keywords] # 将关键词列表转换为小写

    summary_keywords = self.extract_keywords(summary)
    summary_keywords = ' '.join(summary_keywords)
    # 将关键词组合成一个列表
    sentence_keywords = ' '.join(keywords)
    documents = [sentence_keywords, summary_keywords]
    # 创建一个TF-IDF向量化器
    vectorizer = TfidfVectorizer()
    # 将关键词转换为TF-IDF特征向量
    tfidf_matrix = vectorizer.fit_transform(documents)
    # 计算余弦相似度
    similarity_matrix = cosine_similarity(tfidf_matrix[0], tfidf_matrix[1])
    # 提取相似度值
    match_rate = similarity_matrix[0][0]
    return match_rate
```

之后我们定义了一个 get_results 函数来获得最终给前端的返回结果, 我们首先遍历本地数据, 找到与 query 匹配的 token_counts > 0 的 json 项, 并按照 token_counts 进行排序, 获取前 100 个对象。同时如果匹配对象过少则按一定计算方式直接返回查找到的结果, 若有足够的匹配数量则进行下一步操作: 我们继续针对 token_counts 计算匹配率的权重, 创建一个权重列表 weights

(其中的元素是根据每个对象的 `token_counts` 计算出的权重值) 并处理得到一个归一化后的权重列表 `normalized_weights`。之后调用 `calculate_match_rate` 计算关键词和摘要之间的匹配率得到 `match_rate`，最后综合考虑 `match_rate` 和 `token_counts` 的权重，计算最终的匹配率 (`matchRate`)。根据匹配率 (`matchRate`) 进行排序，获取前三个结果返回给前端。

```
def get_results(self, query):
    if not self.data:
        raise ValueError("Data not loaded. Please load the data using 'load_data' method.")

    keywords = self.extract_keywords(query)
    print(keywords)

    # 创建一个列表来保存结果
    results = []

    # 遍历数据，找到最token_counts > 0 的 token_counts 的 obj
    top_objs = []
    for obj in self.data:
        token_counts = self.count_list_occurrences(obj, keywords)
        if token_counts > 0:
            top_objs.append((obj, token_counts))

    # 根据 token_counts 进行排序，获取最大的 100 个对象
    sorted_objs = sorted(top_objs, key=lambda x: x[1], reverse=True)[:100]
    if len(sorted_objs) <= 3:
        for obj, token_counts in sorted_objs:
            result = {
                'title': obj["Second headline"],
                'summary': obj['Summary'],
                'matchRate': "{:.2%}".format(min(1.0, token_counts / (len(keywords) * 9.6))),
                'url': obj['Url']
            }
            results.append(result)
        return results

    # 计算匹配率权重
    max_token_counts = max([count for _, count in sorted_objs])
    weights = [math.exp(token_counts / max_token_counts) for _, token_counts in sorted_objs]
    total_weight = sum(weights)
    normalized_weights = [weight / total_weight for weight in weights]

    # 对最大的 100 个对象进行 calculate_match_rate 并添加到结果列表
    for (obj, token_counts), weight in zip(sorted_objs, normalized_weights):
        match_rate = self.calculate_match_rate(keywords, obj['Summary'])
        # 综合考虑 match_rate 和 token_counts 权重，并确保最终的 matchRate 不超过
```

```

100%
match_rate = min(1.0, match_rate*weight*163) if match_rate > 0 else min
(1.0, weight*3.5)

result = {
    'title': obj["Second headline"],
    'summary': obj['Summary'],
    'matchRate': "{:.2%}".format(match_rate),
    'url': obj['Url']
}
results.append(result)

# 根据 matchRate 进行排序, 获取前三个结果
sorted_results = sorted(results, key=lambda x: float(x['matchRate']).rstrip('%')
), reverse=True)[:3]
return sorted_results

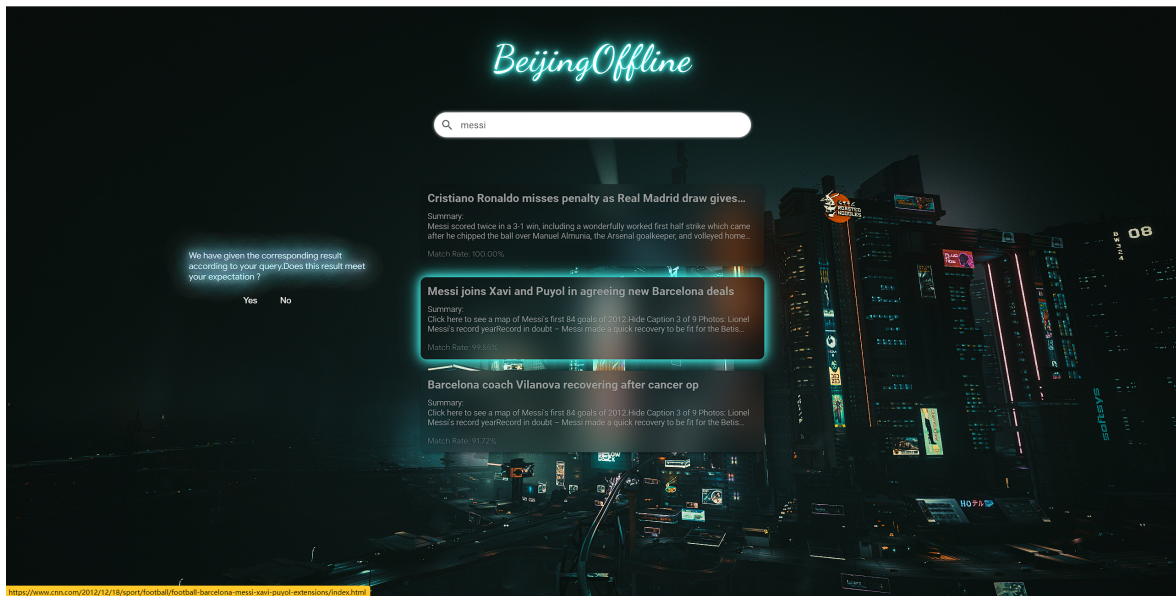
```

3.5 扩展要求实现

多媒体信息抽取：在 CNN News 中通过爬取网页中图片的描述信息（Caption：“xxxxxx”）来获取得到图片的相应信息，并将图片中的信息作为信息抽取的考量范围来影响最终的匹配成果。

4 运行示例

最终界面如下图所示，再搜索栏中输入要查询的词汇，后端收到查询的词汇返回最为相关的几条内容，并在前端中分条展示各个 news 的标题以及我们分析得到的 summary 以及计算的出的 match rate，由于在前端中展示 url 过于紊乱，因此在悬停到对应的选项卡时会看到对应的 url 并在点击选项卡后可以跳转到对应的页面。同时我们引入了人工评价，在查询到相关内容之后，屏幕左侧会出现相应的 feedback，用户可以在这里针对系统查询到的内容提供人工反馈。



参考文献

- [1] Alec Radford et al. "Language models are unsupervised multitask learners". In: *OpenAI blog* 1.8 (2019), p. 9.